Lab 6: Code and Branch Coverage Analysis

Student & Repository Details

SRN: PES1UG24AM396 | **Assigned Problem:** #51 — Self Storage (SETAPESU26/L6_L7_51_self_storage)
Submission Repository: Samarth-S-Maladkar/PES1UG24AM396_SE_LAB

1. Module Under Test & Initial Coverage Baseline

The assigned module Lab 6/self_storage.py contains **37 executable statements** and **26 decision branches** across three functions: `rent_due(unit_size, months, is_climate, is_member)`, `access_allowed(hour, visitors, is_owner)`, and `payment_status(days_to_due, is_autopay, has_prior_late)`. Running coverage on the starter Lab 6/test_self_storage.py executes only 14 of 37 statements (38% line coverage, 32% branch coverage), leaving lines `6, 8, 11, 15-23, 28, 30, 35-45` untested.

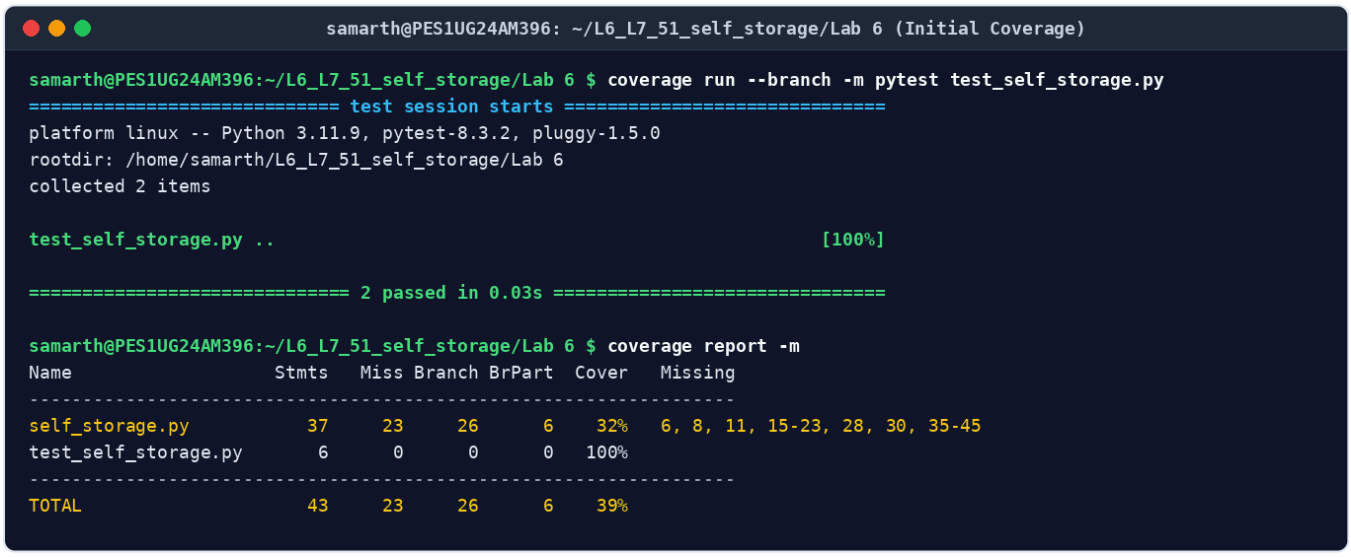


Figure 1: Step 4 — Initial coverage run on starter test_self_storage.py showing 32% branch coverage (23 missed statements).

2. Code Coverage Analysis (>90% Target: 100% Achieved)

In `CodeCoverageTest_self_storage.py`, 11 targeted unit tests were designed to execute every statement in `self_storage.py` at least once, including exception paths (unknown unit size and negative months), member bonus coverage, climate-controlled long-term discounts, owner/non-owner access rules, and all three payment status tiers (`current`, `grace`, and `lien`).

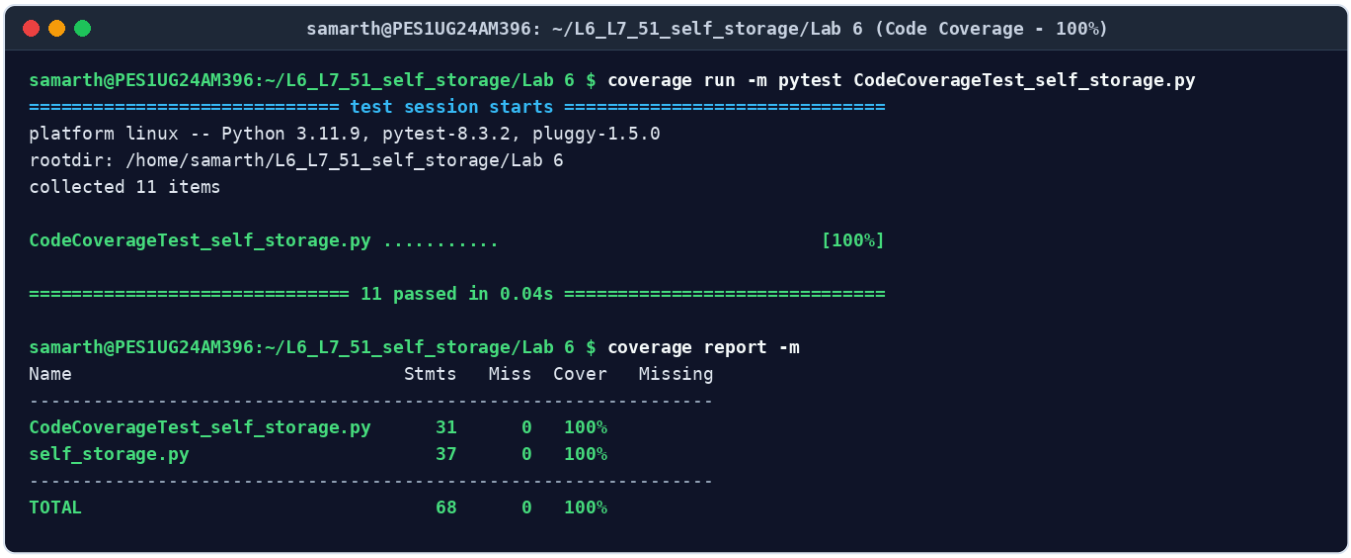


Figure 2: Step 5 — Terminal output for CodeCoverageTest_self_storage.py achieving 100% statement (line) coverage (37/37 statements).

3. Branch Coverage Analysis (>90% Target: 100% Achieved)

While `CodeCoverageTest_self_storage.py` achieves 100% line coverage, several single-sided `if` conditions require both `True` and `False` outcomes to achieve full branch coverage (such as `is_member and unit_size != "large"` when `unit_size == "large"`, `extra > 6` when `is_climate` is `False`, and `has_prior_late` when `False`). In `BranchCoverageTest_self_storage.py`, 16 comprehensive test cases exercise all 26 decision branches.

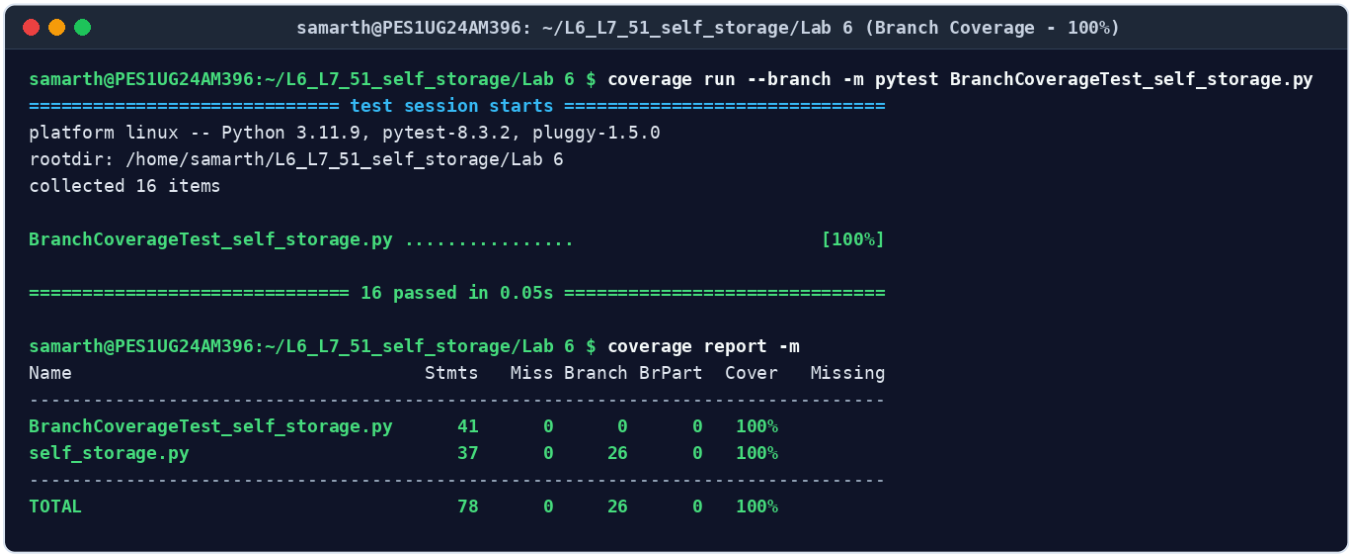


Figure 3: Step 7 — Terminal output for BranchCoverageTest_self_storage.py achieving 100% branch coverage (37 statements, 26 branches, 0 missed).

Test Suite Stage	Test File	Stmts	Miss	Branches	Coverage
Initial Starter Suite	`test_self_storage.py`	37	23	6 / 26	32%

Test Suite Stage	Test File	Stmts	Miss	Branches	Coverage
Step 5: Line Coverage Suite	`CodeCoverageTest_self_storage.py`	37	0	—	100%
Step 7: Branch Coverage Suite	`BranchCoverageTest_self_storage.py`	37	0	26 / 26	100%

4. Test Case Implementations

4.1 CodeCoverageTest_self_storage.py

```
import unittest
from self_storage import rent_due, access_allowed, payment_status

class TestSelfStorageCodeCoverage(unittest.TestCase):
    def test_within_deposit_months(self):
        self.assertEqual(rent_due("small", 1, False, False), 0)

    def test_unknown_unit_size_raises(self):
        with self.assertRaises(ValueError):
            rent_due("extra_large", 3, False, False)

    def test_negative_months_raises(self):
        with self.assertRaises(ValueError):
            rent_due("small", -1, False, False)

    def test_member_climate_long_term(self):
        self.assertEqual(rent_due("medium", 11, True, True), 840)

    def test_non_member_non_climate_short_term(self):
        self.assertEqual(rent_due("small", 4, False, False), 240)

    def test_access_allowed_owner(self):
        self.assertTrue(access_allowed(2, 3, True))

    def test_access_denied_after_hours(self):
        self.assertFalse(access_allowed(23, 1, False))

    def test_access_allowed_regular_hours(self):
        self.assertTrue(access_allowed(12, 1, False))

    def test_payment_status_autopay_prior_late_current(self):
        self.assertEqual(payment_status(12, True, True), "current")

    def test_payment_status_manual_grace(self):
        self.assertEqual(payment_status(5, False, True), "grace")
```

```
    def test_payment_status_lien(self):
        self.assertEqual(payment_status(-20, False, True), "lien")
```

4.2 BranchCoverageTest_self_storage.py

```
import unittest
from self_storage import rent_due, access_allowed, payment_status

class TestSelfStorageBranchCoverage(unittest.TestCase):
    def test_unknown_unit_size_raises(self):
        with self.assertRaises(ValueError):
            rent_due("invalid_size", 2, False, False)

    def test_negative_months_raises(self):
        with self.assertRaises(ValueError):
            rent_due("small", -2, False, False)

    def test_within_deposit_months(self):
        self.assertEqual(rent_due("small", 1, False, False), 0)

    def test_member_large_unit_no_extra_covered_month(self):
        self.assertEqual(rent_due("large", 5, False, True), 160)

    def test_member_small_unit_climate_short_extra(self):
        self.assertEqual(rent_due("small", 5, True, True), 360)

    def test_climate_long_extra_discount(self):
        self.assertEqual(rent_due("small", 9, True, False), 840)

    def test_non_climate_long_extra_discount(self):
        self.assertEqual(rent_due("small", 9, False, False), 560)

    def test_access_owner_within_limit(self):
        self.assertTrue(access_allowed(1, 4, True))

    def test_access_owner_exceeds_limit(self):
        self.assertFalse(access_allowed(14, 5, True))

    def test_access_non_owner_early_morning_denied(self):
        self.assertFalse(access_allowed(5, 1, False))

    def test_access_non_owner_late_night_denied(self):
        self.assertFalse(access_allowed(23, 1, False))

    def test_access_non_owner_valid_hours_allowed(self):
        self.assertTrue(access_allowed(12, 2, False))

    def test_access_non_owner_too_many_visitors(self):
        self.assertFalse(access_allowed(12, 3, False))
```

```
    def test_payment_autopay_no_prior_late_current(self):
        self.assertEqual(payment_status(0, True, False), "current")

    def test_payment_manual_prior_late_grace(self):
        self.assertEqual(payment_status(0, False, True), "grace")

    def test_payment_manual_no_prior_late_lien(self):
        self.assertEqual(payment_status(-11, False, False), "lien")
```